

Project report

Topic: Flexfit Studio

(Gym Management and Membership Operations Application)

BY:

JAIRAJ A,

MCA 1ST YEAR,

**DAYANANDA SAGAR COLLEGE OF ARTS
SCIENCE AND COMMERCE.**

ABSTRACT

FlexFit Studio is a comprehensive gym management application designed to support the day-to-day operations of a modern fitness facility. The system provides members with capabilities such as browsing and booking fitness classes, purchasing memberships, utilizing class credits, and joining waitlists. It also supports staff operations, including front-desk management, trainer administration, payment and membership handling, and revenue reporting. In addition, the platform provides corporate functionality that allows companies to purchase credit pools for their employees to use for eligible fitness services.

The project is implemented using **Next.js 15 with the App Router, TypeScript, tRPC, Drizzle ORM, SQLite, and Tailwind CSS**. The primary objective of the development task is to refactor and restructure an existing application that has evolved through multiple development teams while preserving its existing behavior. The focus is therefore not on changing the user-facing functionality, but on improving code organization, maintainability, reusability, readability, and adherence to modern Next.js and TypeScript development practices.

The refactoring process addresses oversized files, duplicated business logic, tightly coupled responsibilities, and inconsistent code organization. Related functionality is separated into appropriate modules, reusable logic is centralized, and components are structured according to clear responsibilities. Throughout the process, existing inputs, outputs, validation behavior, errors, edge cases, and business workflows are preserved to ensure that the application continues to function exactly as expected.

Table of Contents

1. Introduction
2. Organization Case and Project Context
3. Existing System Overview
4. Objectives
5. Scope
6. Technology Stack
7. System Architecture
8. Functional Modules
9. Functional Requirements
10. Non-Functional Requirements
11. Hardware and Software Requirements
12. Refactoring and Modernization Strategy
13. Behavior Preservation Strategy
14. Testing and Validation
15. Security, Privacy, and Data Integrity
16. Observations, Risks, and Engineering Decisions
17. Expected Results
18. Limitations
19. Future Enhancements
20. Conclusion
21. References

1. Introduction

Gym operations involve several connected workflows: a member chooses and attends classes, uses membership benefits or class credits, may enter a waitlist when a class is full, and may transact through a front-desk workflow. Staff need a different perspective, including trainer administration and revenue reporting. Corporate customers add another layer because they can purchase credit pools for employees. A useful gym management system therefore has to coordinate scheduling, people, entitlements, payments, and operational reporting without exposing internal complexity to staff or members.

FlexFit Studio addresses this business problem through a web application built with a modern TypeScript-first stack. The engineering assignment is distinctive because the application already works and the principal task is to make its internal structure more maintainable while keeping the external behavior stable.

2. Organization Case and Project Context

The organization describes FlexFit Studio as a gym management application that has been worked on by five developers over approximately two years without sustained coordination. The resulting engineering challenge is representative of a legacy codebase: functionality exists, but code boundaries, repeated logic, and file responsibilities may have evolved organically rather than through a consistent architecture.

- ✓ Clone the repository rather than fork it and push the completed work to a private repository.
- ✓ Keep the application behavior exactly the same after the refactor.
- ✓ Preserve existing inputs, outputs, errors, and edge cases.
- ✓ Identify what the application currently does instead of relying on a pre-written feature checklist.
- ✓ Break up files that have become too large or contain unrelated responsibilities.
- ✓ Centralize repeated business logic rather than maintaining multiple divergent implementations.
- ✓ Make architectural decisions that are understandable and defensible even if there is no single prescribed folder layout.
- ✓ When a defect is found, either repair it carefully or document it clearly and leave it unchanged when changing behavior would be risky.

3. Existing System Overview

At the business level, the system can be understood as a set of related domains rather than a single undifferentiated application. Members interact primarily with classes, memberships, credits, and waitlists. Staff interact with member records, front-desk actions, trainers, and reports. Corporate customers interact with credit pools that support employee usage. Financial and entitlement operations connect these domains and therefore require especially careful handling during refactoring.

Domain	Primary Responsibilities	Key Business Data
Member operations	Booking, membership usage, credits, waitlists	Members, memberships, class bookings, credits, waitlist entries
Class scheduling	Classes and participation capacity	Classes, schedules, capacity, instructors, attendance/bookings
Front desk	Operational member assistance and transactions	Member check-in/transactions, bookings, payments
Trainer management	Maintain trainer information and assignments	Trainer profiles, schedules/assignments
Revenue reporting	Operational and financial summaries	Payments, purchases, refunds, reporting aggregates
Corporate accounts	Employer-funded class credit pools	Companies, employees, credit pools, allocations/usage

4. Objectives

- ✓ Understand the behavior of the existing application before restructuring it.
- ✓ Introduce clear boundaries between UI, server procedures, database access, and shared business rules.
- ✓ Reduce repeated logic and eliminate conflicting copies of the same rule.
- ✓ Break large files into focused modules with single, understandable responsibilities.
- ✓ Preserve observable application behavior across successful and failure paths.
- ✓ Improve readability, testability, onboarding, and future maintainability.
- ✓ Maintain the existing user-facing experience unless a change is required to preserve correctness or accessibility.
- ✓ Produce a codebase whose architecture can be explained using explicit engineering decisions rather than arbitrary folder conventions.

5. Scope

The project scope includes the application code, database access layer, server-side procedures, shared domain logic, UI component organization, validation boundaries, and regression testing necessary to complete a safe refactor.

In Scope	Out of Scope / Not Required
Refactoring TypeScript and Next.js application structure	Redesigning the entire product UI for visual novelty
Centralizing repeated business logic	Changing business rules merely to make code simpler
Improving module boundaries and responsibility separation	Replacing SQLite solely because another database is more scalable
Protecting working workflows with regression tests	Adding unrelated product features
Documenting defects and architectural decisions	Claiming new requirements that are not part of the assignment
Improving maintainability and developer ergonomics	Changing external behavior without justification

6. Technology Stack

Technology	Role in the System	Reason for Use
Next.js 15 (App Router)	Web application framework, routing, server/client component model	Provides a unified React application structure and server-side capabilities.
TypeScript	Application language and type system	Improves safety, readability, refactoring confidence, and contract clarity.
tRPC	Typed application/server procedures	Enables end-to-end TypeScript contracts between client calls and server procedures.
Drizzle ORM	Database schema and query layer	Provides a TypeScript-oriented relational data-access layer with SQL-level control.
SQLite	Relational persistence	Lightweight embedded relational storage suited to local/self-contained deployments.
Tailwind CSS	UI styling	Utility-first styling within the existing application stack.

Drizzle describes itself as a TypeScript ORM that supports SQLite and provides typed schema/query capabilities; its SQLite documentation also shows direct integration patterns with Next.js App Router. SQLite is an embedded, serverless, zero-configuration relational database engine. These characteristics align well with a compact full-stack application and make preserving the existing database technology a reasonable engineering choice when the assignment does not require a database migration.

7. System Architecture

The recommended architecture for the refactored system is a layered, domain-oriented structure within the existing Next.js application. The important principle is separation of responsibility rather than a particular folder naming convention.

Layer	Responsibility	Examples
Presentation layer	Pages, route composition, reusable UI components, form state	Member views, staff dashboards, booking forms, dialogs
Application/API layer	Typed tRPC procedures, input validation, authorization boundaries	Book class, cancel booking, purchase membership, refund payment
Domain/service layer	Business rules and reusable operations	Credit deduction, waitlist promotion, entitlement checks, refund policy
Data-access layer	Drizzle schema and queries	Members, classes, bookings, payments, credits, companies
Infrastructure/configuration	Database initialization, environment configuration, shared utilities	SQLite connection, configuration, logging/error helpers

Conceptual request flow: User Interface → tRPC procedure → validation/authorization → domain service → Drizzle query → SQLite → domain result → tRPC response → UI state/update. Keeping the domain service independent from page components reduces duplication and makes the same rule reusable across member, staff, and administrative workflows.

8. Functional Modules

8.1 Member Management

Members are the primary end users. The application must maintain member information and connect it to entitlements, bookings, credits, and transaction history.

8.2 Class Booking

Members can select available classes and book them. Booking operations must enforce the existing capacity, eligibility, and credit/membership rules without changing their current semantics.

8.3 Memberships

Members can purchase and use memberships according to the rules already implemented by the application.

8.4 Class Credits

Credits represent a consumable entitlement. Any deduction, restoration, or balance check is a shared business rule and should be implemented centrally during refactoring.

8.5 Waitlists

When class demand exceeds capacity, members can enter a waitlist. Promotion or removal behavior must remain consistent with the current application behavior.

8.6 Front Desk Operations

Staff need an operational interface for member assistance and day-to-day studio transactions.

8.7 Trainer Management

Staff can manage trainers and the information needed to associate trainers with studio activities.

8.8 Revenue Reports

Staff can inspect financial reporting derived from relevant transactions, with calculations kept consistent across report views.

8.9 Corporate Credit Pools

Companies can purchase credit pools that employees can use. This introduces an additional entitlement source that must remain distinct from ordinary personal member credits.

8.10 Administrative Operations

Where supported by the existing system, administrative functions such as refunds and management actions should remain behaviorally unchanged.

9. Functional Requirements

ID	Requirement
FR-01	The system shall authenticate and authorize users according to the roles and access rules implemented by the existing application.
FR-02	The system shall allow eligible members to browse and select available classes.
FR-03	The system shall allow eligible members to book classes while enforcing the existing capacity and entitlement rules.
FR-04	The system shall consume class credits or membership entitlements according to the existing booking rules.
FR-05	The system shall allow members to join and leave waitlists according to existing class-capacity rules.
FR-06	The system shall preserve the existing waitlist promotion and cancellation behavior.
FR-07	The system shall support membership purchase and management workflows already provided by the application.
FR-08	The system shall support staff front-desk workflows supported by the current application.
FR-09	The system shall support trainer management operations provided by the application.
FR-10	The system shall generate revenue information using the existing business calculations.
FR-11	The system shall support company credit pools and employee usage according to the existing rules.
FR-12	The system shall preserve supported administrative operations, including transaction/refund workflows where implemented.
FR-13	The system shall validate procedure inputs before executing business operations.
FR-14	The system shall return the same meaningful errors and edge-case outcomes after refactoring.
FR-15	The system shall permit repeated execution of supported workflows without requiring application restart.

10. Non-Functional Requirements

Attribute	Requirement
Usability	The existing user workflows should remain easy to understand and should not require unnecessary additional steps after refactoring.
Performance	Common page loads and interactive operations should remain responsive; refactoring should not introduce avoidable database queries or client-side work.
Reliability	Repeated valid operations should execute consistently, and invalid operations should fail through controlled error paths.
Maintainability	Files should have focused responsibilities, shared rules should be centralized, and dependencies should have clear direction.
Type Safety	TypeScript types should accurately represent input, output, and domain data wherever possible.
Testability	Business rules should be sufficiently isolated that they can be tested without rendering entire application pages.
Security	Authorization checks should occur on trusted server-side boundaries; client-side checks alone must not determine access.
Data Integrity	Credits, bookings, memberships, payments, refunds, and corporate allocations must remain internally consistent.
Compatibility	The refactored system must continue to support the existing supported runtime, browser, and database setup.
Observability	Errors and operational failures should be diagnosable without exposing sensitive user or financial data.
Extensibility	New domains or additional workflows should be addable without modifying unrelated modules.

11. Hardware and Software Requirements

Component	Minimum	Recommended
CPU	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5 or better
RAM	8 GB	16 GB
Storage	5 GB free space	10 GB+ SSD free space
OS	Windows 10/11, Linux, or macOS	Windows 11 or current Ubuntu LTS
Node.js	Version compatible with the project lockfile/configuration	Latest project-compatible LTS release
Browser	Modern Chromium/Firefox/Safari browser	Latest Chrome/Edge/Firefox
Database	SQLite supported by the project configuration	Same SQLite setup used by the project
Development tools	Git and a TypeScript-capable IDE	VS Code + Git + terminal
GPU	Not required for normal web application execution	Not applicable / optional

Because the supplied application uses SQLite rather than a separate database server, local setup is comparatively simple. SQLite is designed as a serverless, self-contained database engine, which is appropriate for a small or self-contained application unless deployment requirements later demand a client/server database.

12. Refactoring and Modernization Strategy

The refactor should be incremental and behavior-first. The safest sequence is to first understand the existing behavior, then extract logic without changing semantics, then simplify file structure, and finally validate the complete workflow again.

- ✓ Inventory the repository: identify routes, pages, components, tRPC routers/procedures, Drizzle schema/query modules, utilities, and configuration.
- ✓ Build a behavioral map: record each user-visible workflow, its inputs, expected output, validation, side effects, and error cases.
- ✓ Identify large files and mixed responsibilities: separate UI rendering from data loading, mutation logic, validation, and domain rules.
- ✓ Identify duplicated business logic: especially eligibility checks, credit calculations, booking validation, financial totals, and common entity lookups.
- ✓ Extract pure/shared domain functions first, keeping the original rule semantics unchanged.
- ✓ Move database access behind focused data-access functions where repeated queries or schema details are leaking into multiple features.
- ✓ Keep tRPC procedures thin: validate input, perform authorization, call domain logic/data access, and return the established result/error contract.
- ✓ Reuse UI components only where their responsibilities and props are genuinely shared; avoid creating generic components that hide domain-specific behavior.
- ✓ Run regression tests after each significant extraction instead of waiting until the end.
- ✓ Document intentional non-fixes where changing the behavior would be riskier than leaving a known issue unchanged.

13. Behavior Preservation Strategy

Behavior preservation is the central acceptance criterion. A refactor is successful only when the external contract remains stable while the internal structure improves.

Behavior Area	Preservation Check
Inputs	Forms and mutation calls accept the same valid and invalid values unless a validation bug must be corrected deliberately.
Outputs	Successful operations return the same business outcome and equivalent data.
Errors	Existing error conditions remain distinguishable and occur for the same underlying invalid operations.
Edge cases	Boundary cases such as full classes, insufficient credits, duplicate actions, cancellations, and refunds retain their current semantics.
Side effects	Booking/credit/payment changes occur in the same logical sequence and maintain data consistency.
UI behavior	Existing user-facing workflows, labels, navigation, and interaction expectations should not be unintentionally altered.
Authorization	A user who could perform an action before the refactor should retain that ability, while unauthorized users remain blocked.

A practical safety rule is to treat every observed behavior as part of the current contract until there is explicit evidence that it is unintended. This avoids “cleaning up” a strange edge case and accidentally breaking a workflow the organization expects to keep.

14. Testing and Validation

Testing for this assignment should focus on regression protection rather than only conventional unit-test coverage. The test strategy should demonstrate that the refactor changes code organization, not product behavior.

Test Level	Purpose	Examples
Unit tests	Protect reusable business rules	Credit deduction, eligibility, capacity calculations, refund calculations, waitlist rules
Procedure tests	Protect API/business boundaries	Book class, cancel booking, buy membership, issue/refund transaction
Integration tests	Protect database interactions	Create/update member, booking plus credit mutation, company pool allocation
UI smoke tests	Protect critical user workflows	Member booking flow, waitlist flow, staff operational flow
Regression tests	Compare pre-refactor and post-refactor behavior	Same valid input, same output; same invalid input, same error class/message where required
Manual exploratory tests	Find unrecorded edge cases	Boundary capacities, repeated clicks, cancellations, empty states, unusual dates

Scenario	Expected Outcome
Book a class with available capacity and valid entitlement	Booking succeeds and the appropriate entitlement is consumed according to existing rules.
Book a full class	Existing full-class behavior occurs, such as waitlist eligibility where supported.
Book without valid credits/membership	Operation is rejected with the existing business error.
Cancel a booking	Cancellation follows existing credit/refund/waitlist rules.
Join a waitlist	Waitlist entry is created only when existing conditions permit.
Promote a waitlisted member after a vacancy	Promotion follows the established ordering and eligibility behavior.
Purchase membership	Transaction completes using the same inputs and financial rules.
Refund an eligible payment	Refund is processed according to the existing administrative workflow.
Use company-funded credits	Employee usage is reflected against the correct corporate pool and existing limits.
Run revenue report	Totals remain consistent with the underlying transaction records.

15. Security, Privacy, and Data Integrity

- ✓ Authorization should be enforced on server-side procedures and not trusted solely to client-rendered controls.
- ✓ Validation should occur at the procedure boundary so invalid data is rejected before reaching business logic or the database.
- ✓ Payment and refund operations should preserve transactional consistency and avoid partial updates.
- ✓ Credit balances should not become negative unless the current business rules explicitly allow that state.
- ✓ Corporate credit usage should be attributable to the correct company and employee relationship.
- ✓ Sensitive operational information should not be unnecessarily logged to the browser or server logs.
- ✓ Environment secrets and database files should not be committed to version control.
- ✓ Database migrations should be reviewed as behavior-sensitive changes, especially where existing records are involved.

16. Observations, Risks, and Engineering Decisions

Risk / Observation	Impact	Recommended Treatment
Large mixed-responsibility files	High maintenance cost and difficult testing	Split by responsibility; keep public behavior unchanged.
Repeated business logic	Rules can drift and produce inconsistent outcomes	Create shared domain functions/services.
UI and server logic tightly coupled	Harder regression testing and reuse	Move business rules behind server/domain boundaries.
Database logic repeated across features	Schema/query details leak everywhere	Centralize common access patterns without hiding complex queries unnecessarily.
Legacy edge cases	Refactoring may unintentionally “fix” expected behavior	Capture them in regression tests before changing code.
Broad data-model changes	Migration risk and behavior drift	Only change the schema when the existing design genuinely blocks maintainability or correctness; document the decision.
No single mandated folder layout	Architecture can become arbitrary	Choose boundaries based on domain responsibility and dependency direction.

The organization explicitly allows both conservative refactoring and deeper model changes. A defensible approach is to avoid changing the database schema unless it solves a demonstrable problem that cannot reasonably be solved at the application layer. Drizzle supports typed schema and migrations, so deeper changes are technically feasible, but migration risk should be justified against the behavior-preservation requirement.

17. Expected Results

- ✓ The application continues to support all previously working member, staff, trainer, reporting, and corporate-credit workflows.
- ✓ Large or mixed-purpose modules are divided into smaller units with clear responsibilities.
- ✓ Repeated business rules are implemented in one authoritative location.
- ✓ TRPC procedures become easier to read because business operations are delegated to focused services/functions.
- ✓ Database queries and schema concerns are kept closer to the data-access layer.
- ✓ The UI remains behaviorally stable while becoming easier to maintain.
- ✓ Critical workflows are protected by regression tests that describe real application behavior.
- ✓ The resulting architecture is easier for a new developer to understand and extend.

18. Limitations

The repository at <https://github.com/jai2004raj/flexfit-studio.git> could not be fetched directly from the report-generation environment during preparation because external Git hosting access was unavailable. Consequently, this report does not invent exact filenames, line counts, router names, database table names, screenshots, test counts, or refactor diffs. Statements about the application's modules and technology stack are grounded in the organization-provided project brief. Before academic or organizational submission, file-specific implementation details should be checked against the final repository commit.

This limitation does not affect the suitability of the report as a project-level design, requirements, architecture, testing, and refactoring report, but it does mean that exact implementation evidence belongs in the final appendix once the repository snapshot is available.

19. Future Enhancements

- ✓ Add broader automated end-to-end coverage for the highest-value member and staff workflows.
- ✓ Introduce stronger observability and structured error reporting for production deployments.
- ✓ Consider migration from SQLite to a client/server relational database only if concurrent usage, deployment scale, or reporting requirements justify it.
- ✓ Add role-aware audit trails for financial and administrative operations where required by the business.
- ✓ Improve performance monitoring for reports and high-volume booking operations.
- ✓ Introduce stronger domain-level documentation and diagrams as the application grows.

20. Conclusion

FlexFit Studio is best understood as a behavior-preserving modernization exercise rather than a feature-development project. The core engineering problem is to take a functional application that has accumulated inconsistent structure across multiple development cycles and reshape it into a clear, modular TypeScript/Next.js codebase without changing what users observe.

The recommended solution is a layered, domain-oriented architecture in which UI components focus on presentation, tRPC procedures define typed application boundaries, domain services own business rules, and Drizzle-based data access owns database interaction. The most important quality gate is regression protection: successful and unsuccessful workflows must retain their existing semantics, including edge cases. This approach directly aligns the technical work with the organization's stated evaluation criteria and produces a system that is easier to understand, test, maintain, and extend.

21. References

- ✓ GitHub repository supplied for the assignment: <https://github.com/jai2004raj/flexfit-studio.git>
- ✓ Original assignment repository referenced by the organization: <https://github.com/Rahul-Callus/flexfit-studio>
- ✓ Next.js documentation: <https://nextjs.org/docs>
- ✓ tRPC documentation: <https://trpc.io/docs>
- ✓ Drizzle ORM documentation: <https://orm.drizzle.team/docs>
- ✓ Drizzle ORM GitHub repository and project overview: <https://github.com/drizzle-team/drizzle-orm>
- ✓ SQLite documentation: <https://sqlite.org/docs.html>
- ✓ Tailwind CSS documentation: <https://tailwindcss.com/docs>